

Seasonal Model Fitting to Meteorological Data Using a Damped Gauss–Newton Method

Azra Kurić, Student ID 20113

Faculty of Electrical Engineering, University of Sarajevo
Numerical Optimization—Prof. dr. Izudin Džafić

ABSTRACT

This paper presents a native C++ application for fitting a nonlinear seasonal model to monthly meteorological observations. The model combines an intercept, a linear trend, an annual harmonic, and a semi-annual harmonic. Its six parameters are estimated by minimizing the residual sum of squares with a damped Gauss–Newton method, an analytical residual Jacobian, Gaussian elimination with partial pivoting, and Armijo backtracking. The application is implemented with the natID graphical framework and exposes the complete accepted-iteration history through an interactive dashboard. Experiments on cleaned NOAA records for Sarajevo and Bjelašnica converge in nine and seven accepted iterations, respectively. The final root-mean-square errors are 1.9185°C and 1.8469°C, while the corresponding coefficients of determination are 0.9361 and 0.9274. Application screenshots are used throughout the paper so that the reported visualization exactly matches the submitted software.

Index Terms—Gauss–Newton method, nonlinear least squares, seasonal model, Armijo backtracking, meteorological data, natID, data visualization.

1 INTRODUCTION

Monthly temperature records are dominated by periodic seasonal behavior, but they may also contain a slow long-term trend and smaller deviations from a simple sinusoid. Estimating these components gives a compact mathematical description of the data and provides a clear nonlinear-optimization problem. This project implements that estimation process as a C++20 desktop application. It accepts cleaned monthly records, fits a six-parameter seasonal model, preserves every accepted optimizer state, and visualizes how the fit, residuals, objective value, and parameters change across iterations.

The numerical core is independent of the user interface. The interface is built with natID, a native C++ application framework [5]. The supplied data originate from NOAA monthly station products and were converted to a project-specific CSV schema before fitting [6]. Two stations are included: Sarajevo, with 298 valid observations, and Bjelašnica, with 83 observations. Missing months remain explicit gaps in the month index and are not interpolated.

The main contributions are: 1) a complete derivation and implementation of a damped Gauss–Newton method; 2) an analytical Jacobian for the seasonal model; 3) numerical safeguards including diagonal damping, pivoting, descent verification, and backtracking; and 4) an interactive natID dashboard whose plots are linked to the actual optimizer history.

2 DATA AND SEASONAL MODEL

2.1 Data Preparation

The preprocessing script `clean_data.py` converts the source station files to

```
month, year, month_of_year, temperature_celsius.
```

The optimizer uses the first and fourth fields. The CSV loader checks the header, trims fields, accepts only finite numeric values, ignores malformed rows, and requires at least twelve valid observations. Conversion to optimization vectors additionally enforces strictly increasing month indices. Therefore, duplicate time points and reversed records cannot silently enter the numerical method.

2.2 Parametric Model

For month index t , the predicted temperature is

$$f(t; \theta) = a_0 + a_1 t + A_1 \sin(\omega t + \phi_1) + A_2 \sin(2\omega t + \phi_2), \quad (1)$$

where $\omega = 2\pi/12$ and

$$\theta = [a_0, a_1, A_1, \phi_1, A_2, \phi_2]^\top. \quad (2)$$

Here, a_0 is the baseline temperature, a_1 is the linear change per month, (A_1, ϕ_1) define the annual component, and (A_2, ϕ_2) provide a semi-annual correction. The second harmonic permits a seasonal curve that is not perfectly sinusoidal.

For observation (t_i, y_i) , the residual is

$$r_i(\theta) = y_i - f(t_i; \theta). \quad (3)$$

Positive residuals are observations warmer than the model prediction; negative residuals are cooler. The optimization objective is the residual sum of squares (RSS),

$$F(\theta) = \sum_{i=1}^m r_i(\theta)^2 = \|r(\theta)\|_2^2. \quad (4)$$

The phase parameters make the problem nonlinear. In addition, amplitude–phase pairs have equivalent representations because $A \sin(x + \phi) = (-A) \sin(x + \phi + \pi)$. The implementation normalizes both phases to $[-\pi, \pi)$ after each candidate step without changing the predicted curve.

3 NUMERICAL METHOD

3.1 Gauss–Newton Approximation

Gauss–Newton exploits the least-squares structure of (4) [1, 2]. At iterate θ_k , the residual vector is approximated by

$$r(\theta_k + d) \approx r_k + J_k d, \quad (5)$$

where J_k is the residual Jacobian. Minimizing the squared norm of the linearized residual gives

$$J_k^\top J_k d_k = -J_k^\top r_k. \quad (6)$$

This omits the residual-weighted second-derivative term of the exact Hessian. It is effective when the Jacobian has adequate rank and the current residuals are not excessively large.

3.2 Analytical Jacobian

Let $x_i = \omega t_i + \phi_1$ and $z_i = 2\omega t_i + \phi_2$. Since the implementation differentiates the residual $y_i - f_i$, row i is

$$J_i = [-1, -t_i, -\sin x_i, -A_1 \cos x_i, -\sin z_i, -A_2 \cos z_i]. \quad (7)$$

The RSS gradient is $\nabla F = 2J^\top r$. Evaluating these derivatives analytically avoids finite-difference step selection and makes the sign convention explicit.

3.3 Damping and Linear Solution

To reduce sensitivity to parameter scaling and weakly determined directions, the implemented system is

$$(J_k^\top J_k + D_k) d_k = -J_k^\top r_k, \quad (8)$$

where

$$(D_k)_{jj} = 10^{-6} \max\{1, |(J_k^\top J_k)_{jj}|\}. \quad (9)$$

The six-dimensional dense system is solved by Gaussian elimination with partial pivoting and back substitution. The largest available absolute pivot is selected in every column, and pivots smaller than 10^{-12} produce an explicit singular-system status. Pivoting is a standard defense against avoidable numerical error [4].

3.4 Armijo Backtracking

A full Gauss–Newton step may increase the nonlinear objective when the current parameters are far from the solution. The candidate $\theta_k + \alpha d_k$ is accepted only if

$$F(\theta_k + \alpha d_k) \leq F(\theta_k) + c\alpha \nabla F_k^\top d_k, \quad (10)$$

with $c = 10^{-4}$ [3]. The search starts at $\alpha = 1$ and halves it until acceptance. Failure is reported if α falls below 10^{-10} . The code stores $-J^\top r$, so the directional derivative is evaluated as $-2(-J^\top r)^\top d$ and must be negative.

3.5 Termination and History

Successful termination occurs when

$$\| -J^\top r \|_2 < 10^{-8}, \quad (11)$$

$$\| d \|_2 < 10^{-8}, \quad \text{or} \quad (12)$$

$$\frac{|F_k - F_{k+1}|}{\max(1, |F_k|)} < 10^{-12}. \quad (13)$$

The default iteration limit is 100. Other outcomes distinguish a singular system, failed line search, and maximum-iteration termination. The initial state and every accepted update are stored in aligned parameter and objective histories. Thus, visualization index zero is the exact user-supplied initialization, while every later integer is one accepted update.

Table 1: Final Fit Quality

Station	m	Iter.	RSS ₀	RSS	RMSE	R^2
Sarajevo	298	9	19637.048	1096.800	1.918	0.9361
Bjelašnica	83	7	9178.765	283.125	1.847	0.9274

Table 2: Optimized Parameters

Station	a_0	a_1	A_1	ϕ_1	A_2	ϕ_2
Sarajevo	10.61475	.006434	10.28638	-1.61899	-.61997	2.82965
Bjelašnica	2.30033	-.017913	9.31839	-1.74188	.43552	2.90890

4 IMPLEMENTATION WITH NATID

The project separates numerical and interface responsibilities. `SeasonalModel.h` implements the model, residuals, RSS, Jacobian, norms, updates, and phase normalization. `GaussNewton.h` contains result and status structures, the pivoted solver, damping, backtracking, stopping tests, and histories. `TemperatureData.h` performs CSV validation. The remaining view classes assemble the dashboard, settings, evaluation, navigation, animation, and export behavior.

The `natID` layer uses `gui::Application` and `gui::Window` for the application lifecycle. Views are organized by `gui::StandardTabView` and `gui::GridLayout`; labels, numeric editors, buttons, a station combo box, integer and speed sliders, and a timer provide interaction. The four dashboard panels are drawn on a custom `gui::Canvas` with `gui::Shape` and `gui::DrawableString`.

The Dashboard tab contains: 1) observed temperatures, the selected-iteration curve, and a gray final-fit reference; 2) residual stems, red for warmer and blue for cooler observations; 3) $\log_{10}(\text{RSS})$ against accepted iteration; and 4) current values of all six parameters. Playback changes only the selected history state and redraws the canvas; it does not rerun the optimization. Model Setup keeps initialization and tolerance controls separate from the plots. Final Evaluation reports the stopping reason and summary metrics and exports uniquely named, timestamped CSV files to `SeasonalFit_exports`.

5 EXPERIMENTAL RESULTS

Both station experiments use the default initialization

$$\theta_0 = [10, 0, 5, -3, 2, 2]^\top \quad (14)$$

and the tolerances stated above. The numerical results were obtained by compiling the same optimizer used by the graphical application and running it against the bundled cleaned CSV files.

Both runs terminate through the relative-objective criterion. The iteration counts are generated by the stopping logic, not fixed by station. Sarajevo reduces RSS by 94.41%. The two R^2 values show that the model explains more than 92% of the observed variance, although residual weather variability remains visible in Figs. 1 and 2.

The final RMSE is $\sqrt{\text{RSS}/m}$, and

$$R^2 = 1 - \frac{\sum_i (y_i - f_i)^2}{\sum_i (y_i - \bar{y})^2}. \quad (15)$$

The negative Sarajevo semi-annual amplitude is not problematic: because of amplitude–phase equivalence, its interpretation must include ϕ_2 .

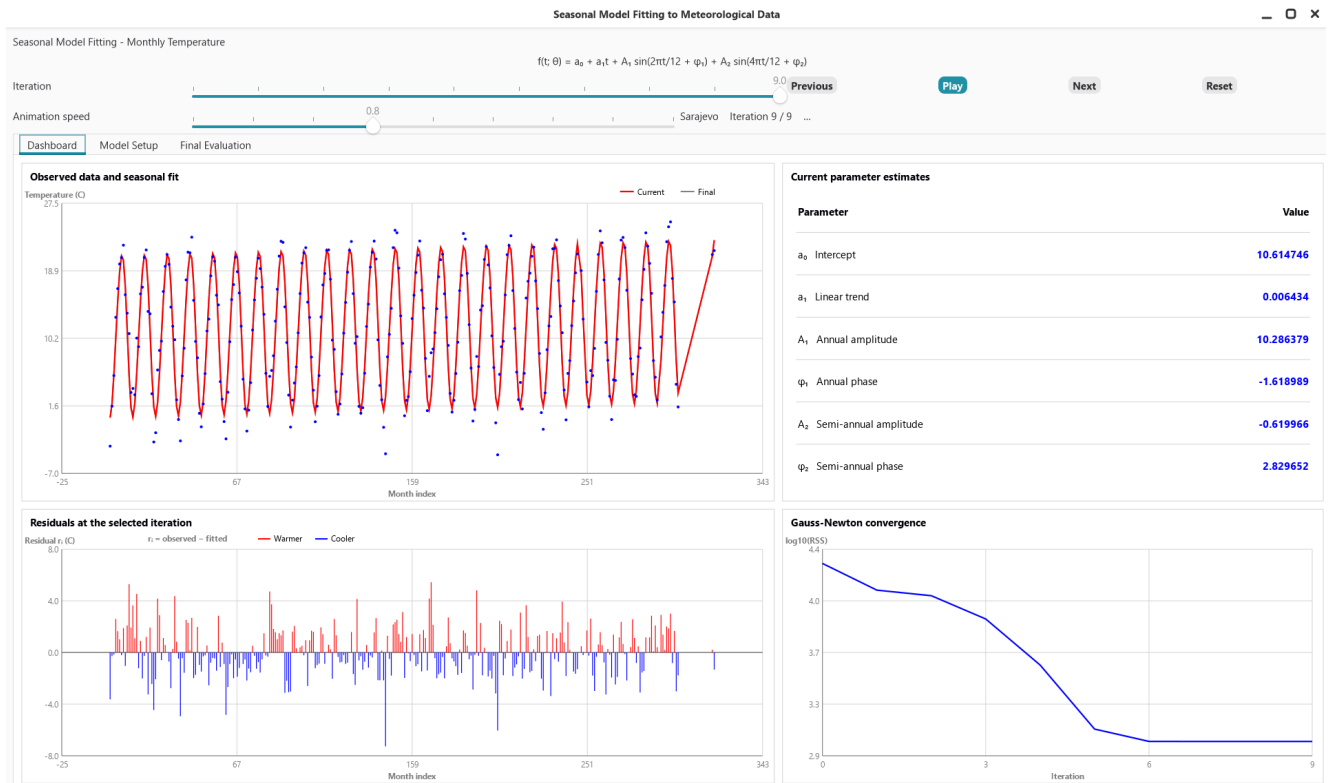


Figure 1: Actual application dashboard for Sarajevo at the final accepted iteration. The upper-left panel displays observations and the fitted seasonal curve; residuals are shown as unmarked red and blue stems, matching the submitted application exactly.

Figure 3(a) illustrates why the history is educationally useful. It does not interpolate artificial fractional iterations: slider value 3 denotes the third accepted parameter update. A deliberately extreme initial intercept, such as $a_0 = 100$, produces a correspondingly extreme iteration-zero plot. Since the intercept enters the model linearly and its residual derivative is constant, the first Gauss–Newton update can remove most of that offset. A large visual change between iterations zero and one is therefore expected behavior.

Figure 3(b) records the evaluation directly from the running program. Each export contains the convergence configuration, initial and final parameters, summary metrics, and one row per observation with its fitted value and residual. The station name, date, time, and milliseconds form a unique filename so that repeated experiments do not overwrite one another.

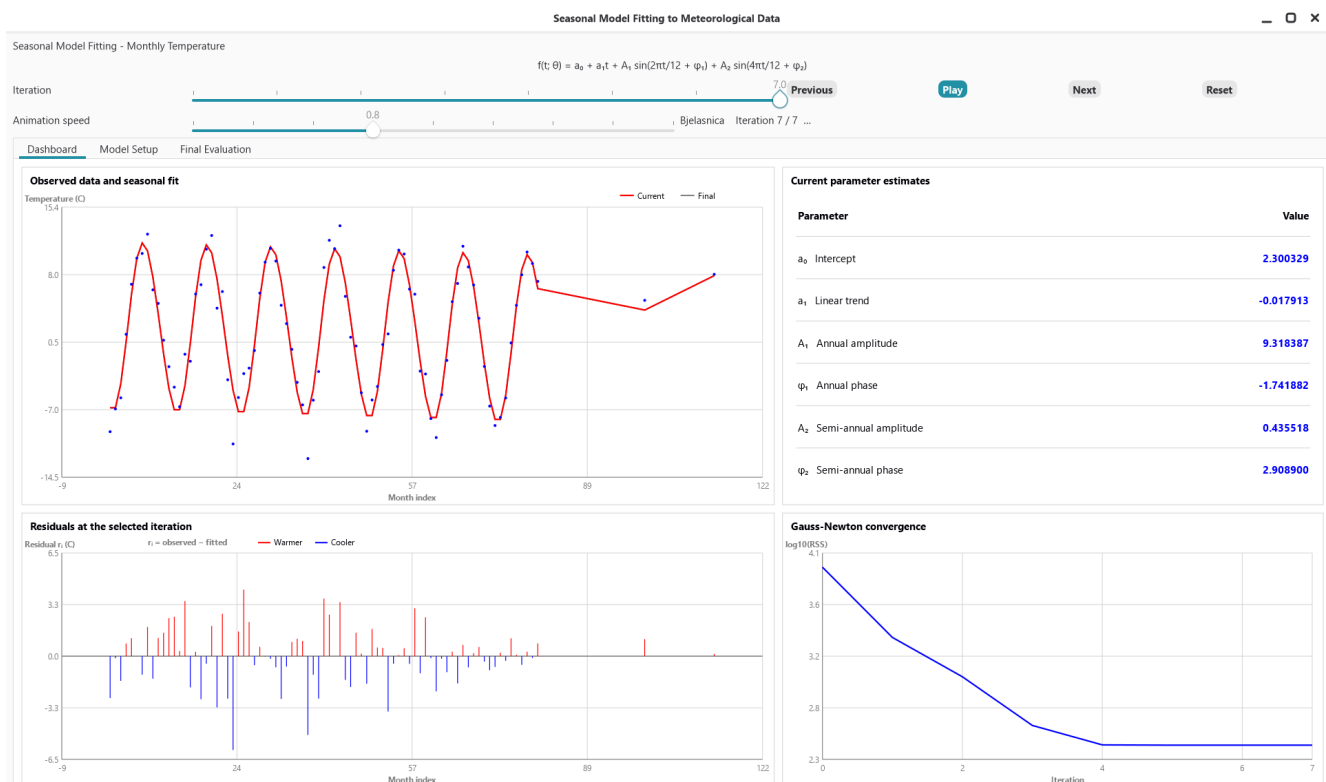
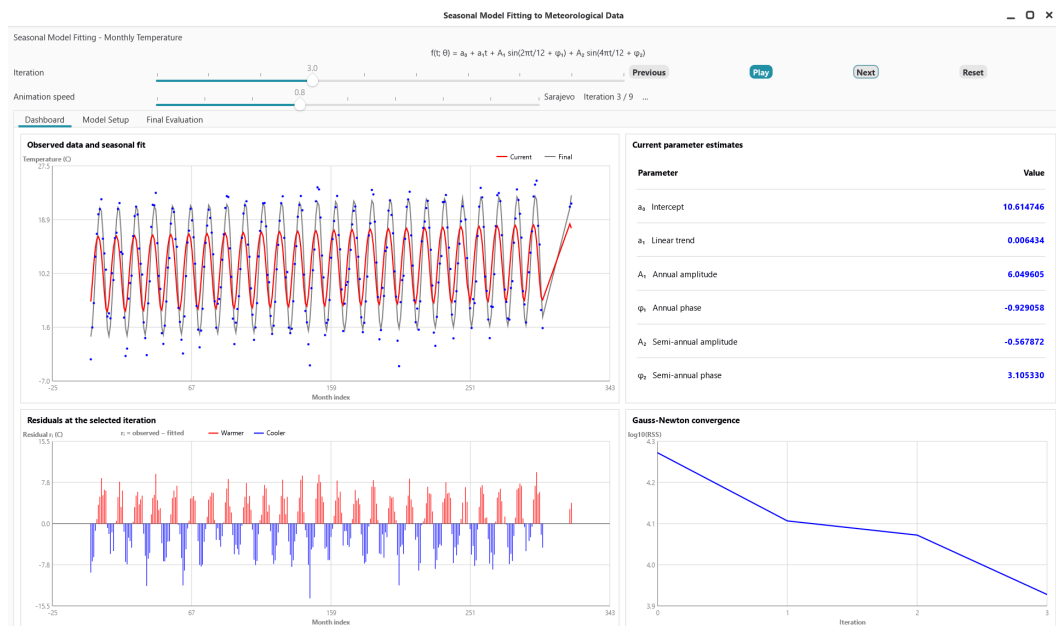
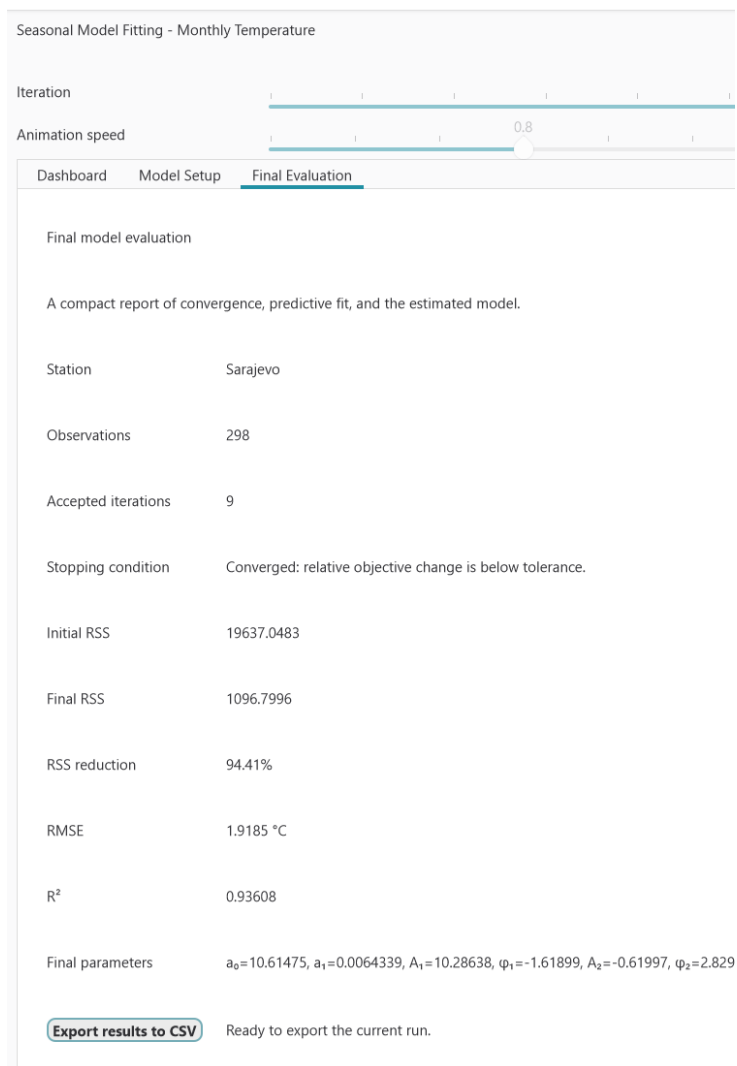


Figure 2: Actual application dashboard for Bjelašnica at the final accepted iteration. Missing calendar months remain gaps in the month index, which explains the long final line segments between isolated observations.



(a) Sarajevo dashboard at accepted iteration 3.



(b) Final Evaluation tab for Sarajevo.

Figure 3: Actual application views used to inspect the optimization history and final result. The screenshots are unmodified apart from cropping unused empty space in (b); no plot, marker, color, or metric has been regenerated.

6 NUMERICAL ROBUSTNESS AND LIMITATIONS

The implementation validates all input dimensions and positive tolerances, uses only finite parsed observations, retains a hard iteration limit, and reports every non-success status. Damping improves the conditioning of the normal equations; partial pivoting and a pivot threshold detect an unusable system; the directional-derivative check rejects a non-descent direction; bounded Armijo backtracking prevents an endless search; and phase normalization maintains readable parameters.

The current model also has limitations. The frequency is fixed to a twelve-month period, the trend is linear, and ordinary least squares gives each observation equal influence. Exceptional months can therefore affect the fit. Furthermore, normal equations square the condition number of J ; QR or singular-value decomposition would be preferable for larger or poorly identifiable models [4]. The application provides descriptive fit metrics, but it does not calculate parameter confidence intervals or test out-of-sample predictions.

Possible extensions include robust or weighted least squares, automatic initialization using a linear sine–cosine representation, Jacobian-based uncertainty estimates, and comparison with Levenberg–Marquardt. These additions would extend the statistical analysis, while the current project remains focused on transparent implementation and visualization of a numerical optimization method.

7 CONCLUSION

This project delivers a complete nonlinear seasonal-fitting workflow in C++20 and natID. A six-parameter trend-plus-harmonics model is optimized by a damped Gauss–Newton method with an analytical Jacobian, pivoted dense solve, Armijo backtracking, phase normalization, and explicit stopping conditions. The interface links every displayed curve, residual stem, objective value, and parameter estimate to a stored optimizer state.

With the default initialization, Sarajevo converges in nine accepted updates and Bjelašnica in seven, reaching RMSE values of approximately 1.92°C and 1.85°C. The IEEE-style report uses screenshots captured from the submitted executable rather than independently regenerated charts, ensuring consistency between the written evaluation and the actual application.

REFERENCES

- [1] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.
- [2] D. M. Bates and D. G. Watts, *Nonlinear Regression Analysis and Its Applications*. New York, NY, USA: Wiley, 1988.
- [3] L. Armijo, “Minimization of functions having Lipschitz continuous first partial derivatives,” *Pacific J. Math.*, vol. 16, no. 1, pp. 1–3, 1966.
- [4] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.
- [5] I. Džafić and contributors, “natID: A native interface development framework,” GitHub repository. [Online]. Available: <https://github.com/idzafic/natID>. Accessed: Sep. 2026.

- [6] NOAA National Centers for Environmental Information, “Global Summary of the Month (GSOM).” [Online]. Available: <https://www.ncei.noaa.gov/access/search/data-search/global-summary-of-the-month>. Accessed: Sep. 2026.